

CLUSTERING

CLUSTERING ALGORITHMS – PART VI

Fabio Stella

Associate Professor

c/o Department of Informatics, Systems and Communication

University of Milano-Bicocca



CLUSTERING ALGORITHMS

The following concepts will be introduced:

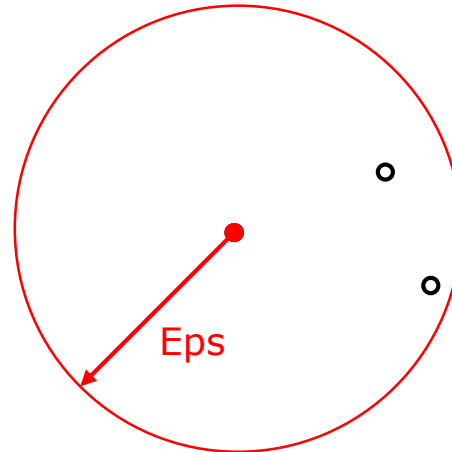
- ✓ **DENSITY**
- ✓ **NEIGHBORHOOD**
- ✓ **CORE POINT, BORDER POINT, NOISE POINT**
- ✓ **DENSITY-BASED CLUSTERING**
- ✓ **DBSCAN**
- ✓ **GRID-BASED/SUBSPACE/KERNEL DENSITY ALGORITHMS**

DENSITY-BASED CLUSTERING TECHNIQUES aim to find dense regions of objects that are surrounded by low-density regions.

DBSCAN is a simple and effective density-based clustering algorithm that illustrates a number of important concepts that are typical of the density-based approach.

Several methods exist to define **DENSITY**, we describe the **CENTER-BASED APPROACH** on which DBSCAN is based.

DENSITY of the red filled circle object is the number of objects within a specific radius (Eps) of that object

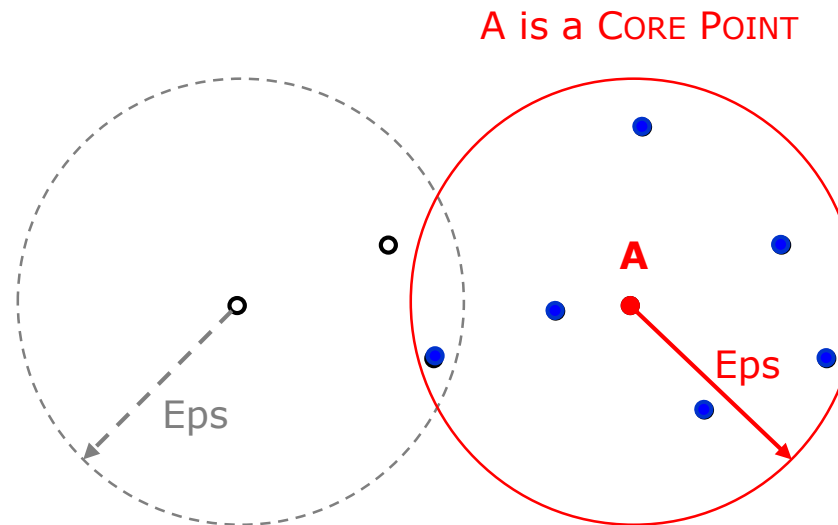


density of any point will depend on the specified radius (Eps).

DENSITY-BASED CLUSTERING TECHNIQUES allow to classify a point (record) as being

- ✓ **CORE POINT**; is in the interior of a density-based cluster. A point is a core point if the number of points within a given neighborhood around the point as determined by the distance function and a user-specified distance parameter, **Eps**, exceeds a certain threshold, **MINPTS**, which is also a user-specified parameter

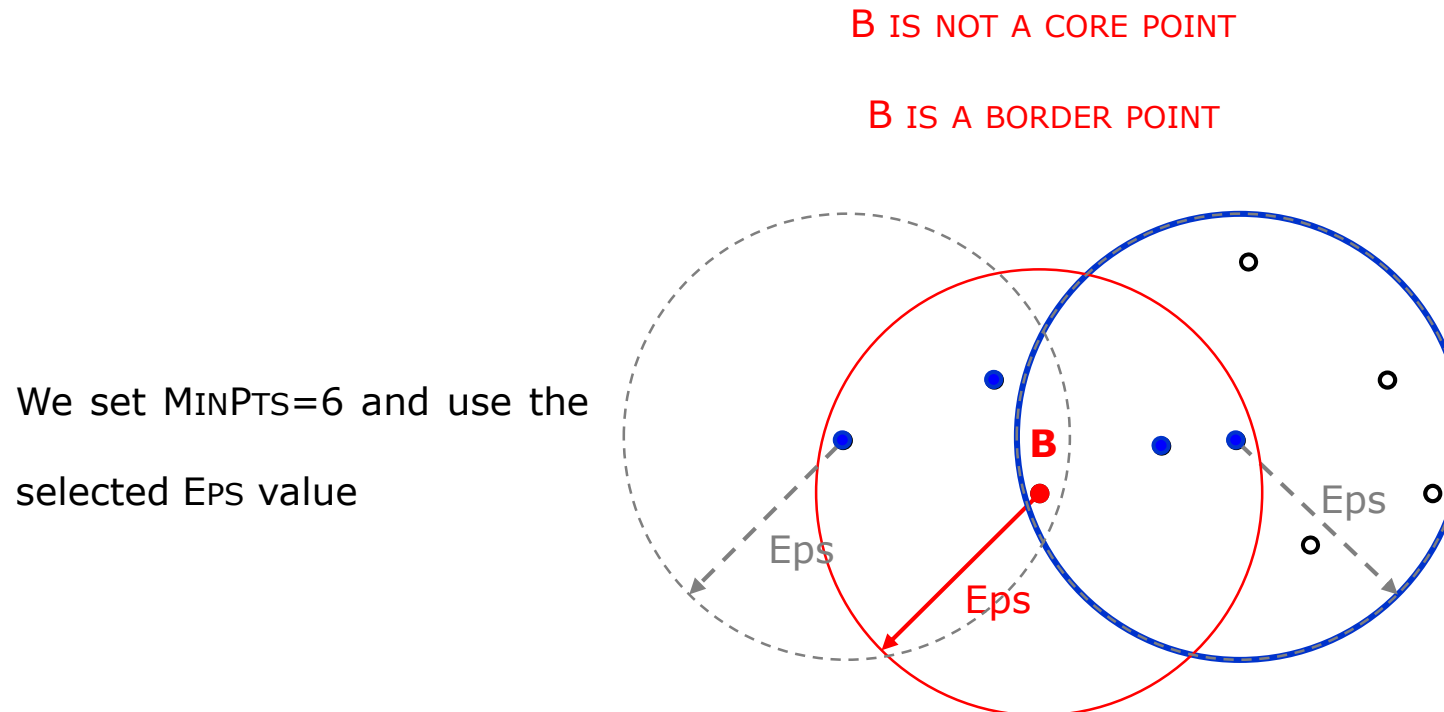
We set **MINPTS=6** and use the selected **Eps** value



DENSITY-BASED CLUSTERING TECHNIQUES allow to classify a point (record) as being

✓ **BORDER POINT;** is not a core point, but falls within the neighborhood of a core point.

A border point can fall in the neighborhood of several core points.

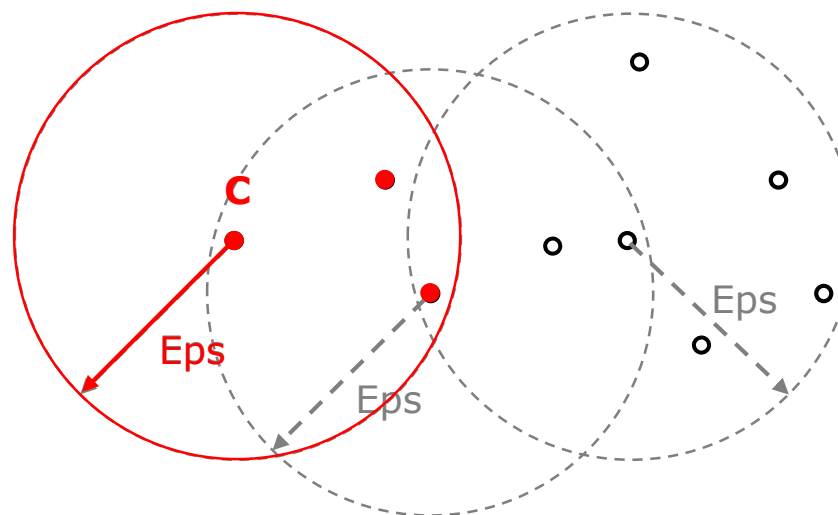


DENSITY-BASED CLUSTERING TECHNIQUES allow to classify a point (record) as being

✓ **NOISE POINT;** is any point that is neither a core nor a border point

C IS NOT A CORE POINT

We set $\text{MINPTS}=6$ and use the
selected EPS value



DENSITY-BASED CLUSTERING TECHNIQUES allow to classify a point (record) as being

✓ **NOISE POINT;** is any point that is neither a core nor a border point

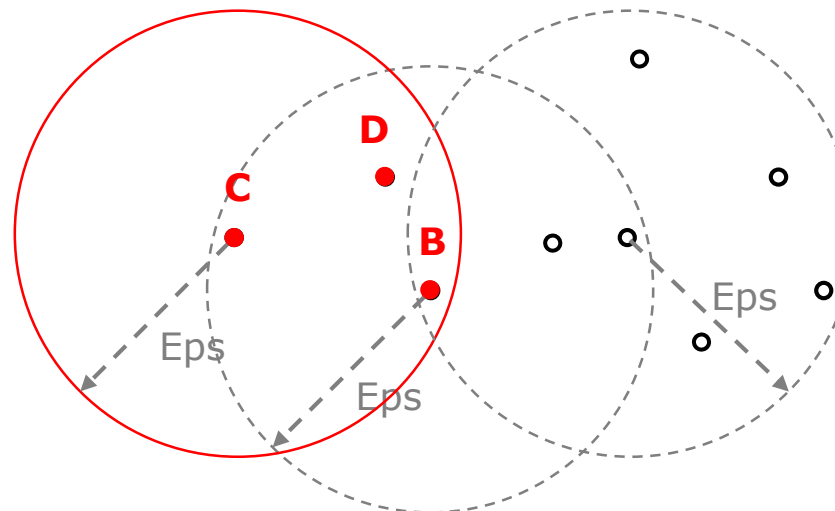
C IS NOT A CORE POINT

NEITHER D NOR B ARE CORE POINTS

C IS NOT A BORDER POINT

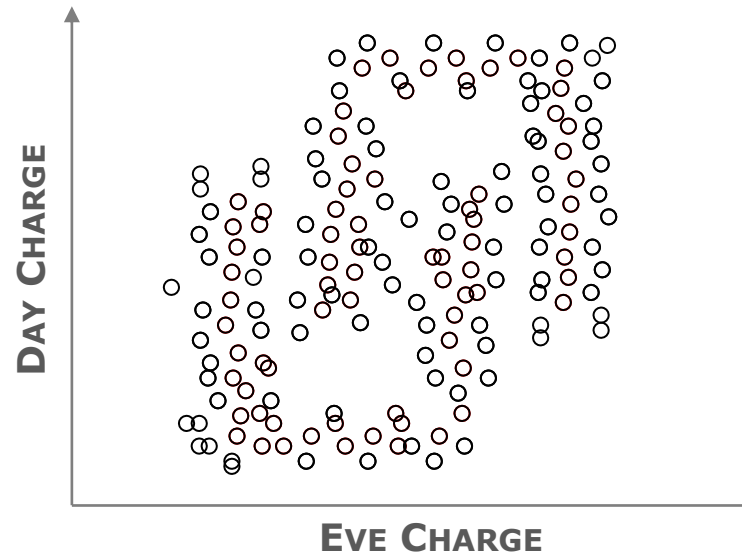
C IS A NOISE POINT

We set $\text{MINPTS}=6$ and use the
selected EPS value



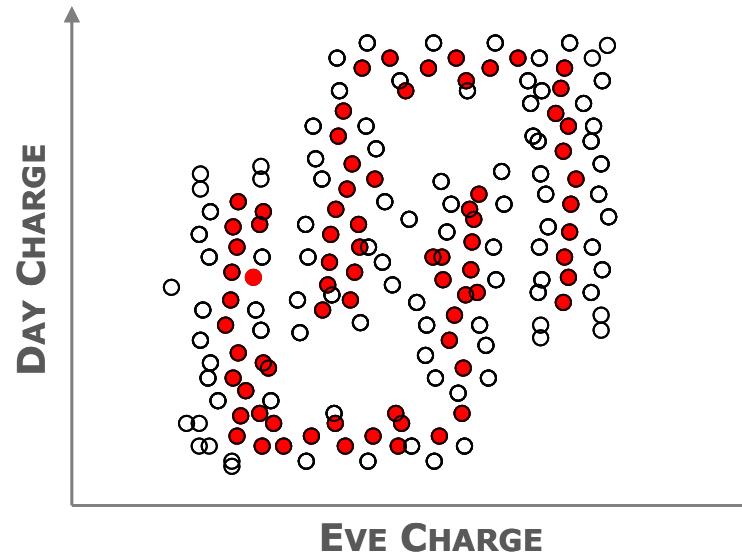
DBSCAN

1. Label all points as core, border, or noise points
2. Eliminate noise points
3. Put an edge between all core points that are within Eps of each other
4. Make each group of connected core points into a separate cluster
5. Assign each border point to one of the clusters of its associated core points



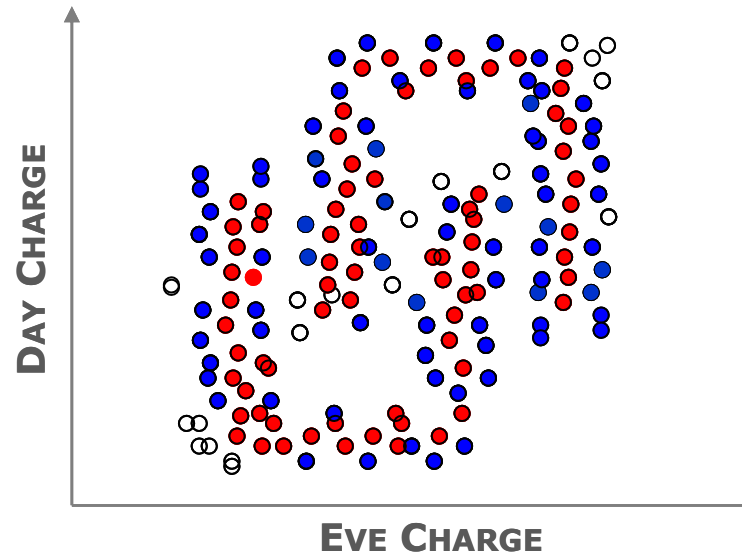
DBSCAN

1. Label all points as core, border, or noise points
2. Eliminate noise points
3. Put an edge between all core points that are within Eps of each other
4. Make each group of connected core points into a separate cluster
5. Assign each border point to one of the clusters of its associated core points



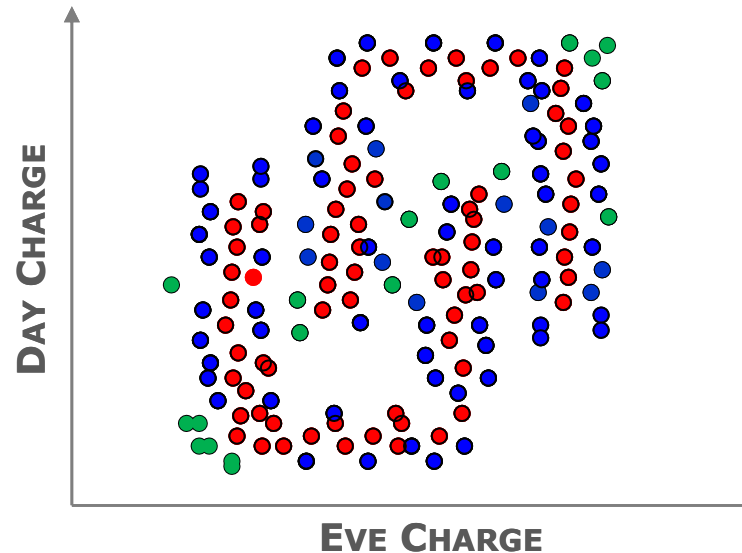
DBSCAN

1. Label all points as core, border, or noise points
2. Eliminate noise points
3. Put an edge between all core points that are within Eps of each other
4. Make each group of connected core points into a separate cluster
5. Assign each border point to one of the clusters of its associated core points



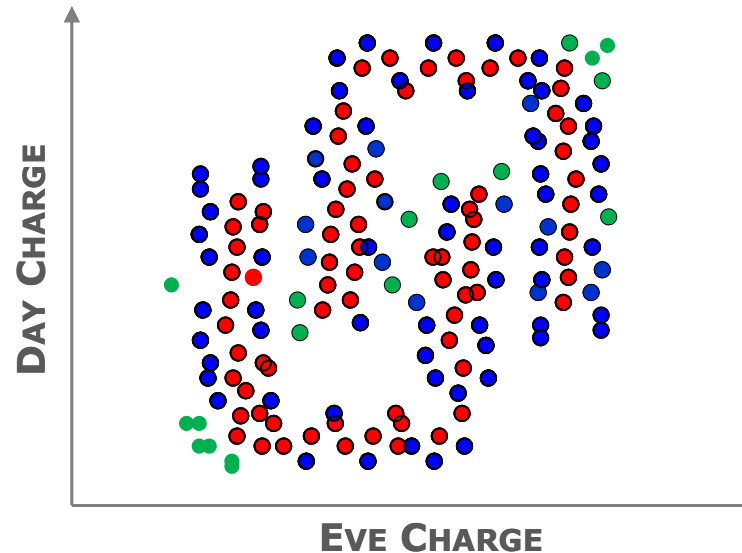
DBSCAN

1. Label all points as core, border, or noise points
2. Eliminate noise points
3. Put an edge between all core points that are within Eps of each other
4. Make each group of connected core points into a separate cluster
5. Assign each border point to one of the clusters of its associated core points



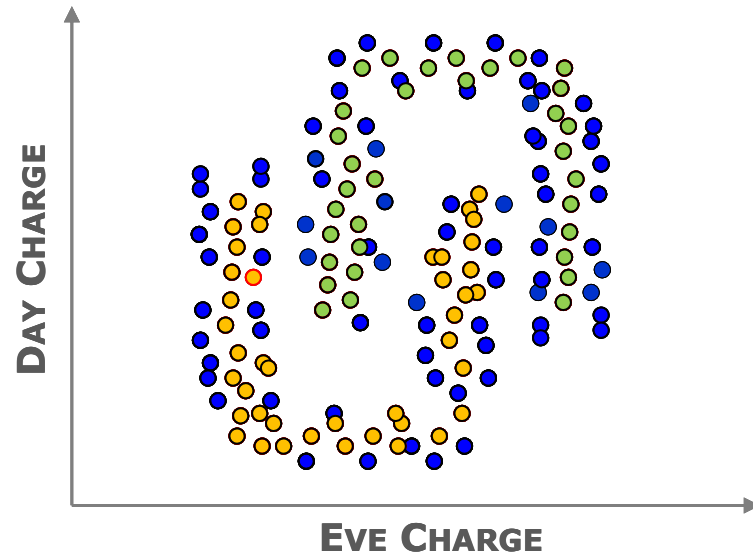
DBSCAN

1. Label all points as core, border, or noise points
2. Eliminate noise points
3. Put an edge between all core points that are within Eps of each other
4. Make each group of connected core points into a separate cluster
5. Assign each border point to one of the clusters of its associated core points



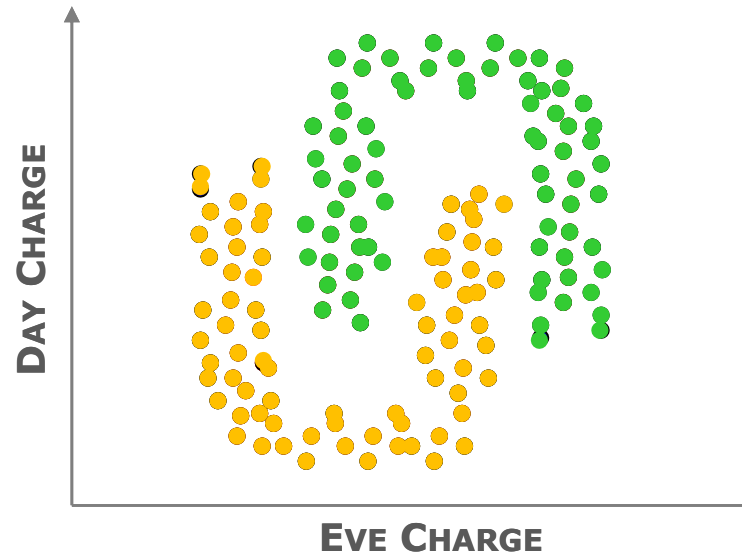
DBSCAN

1. Label all points as core, border, or noise points
2. Eliminate noise points
3. Put an edge between all core points that are within Eps of each other
4. Make each group of connected core points into a separate cluster
5. Assign each border point to one of the clusters of its associated core points

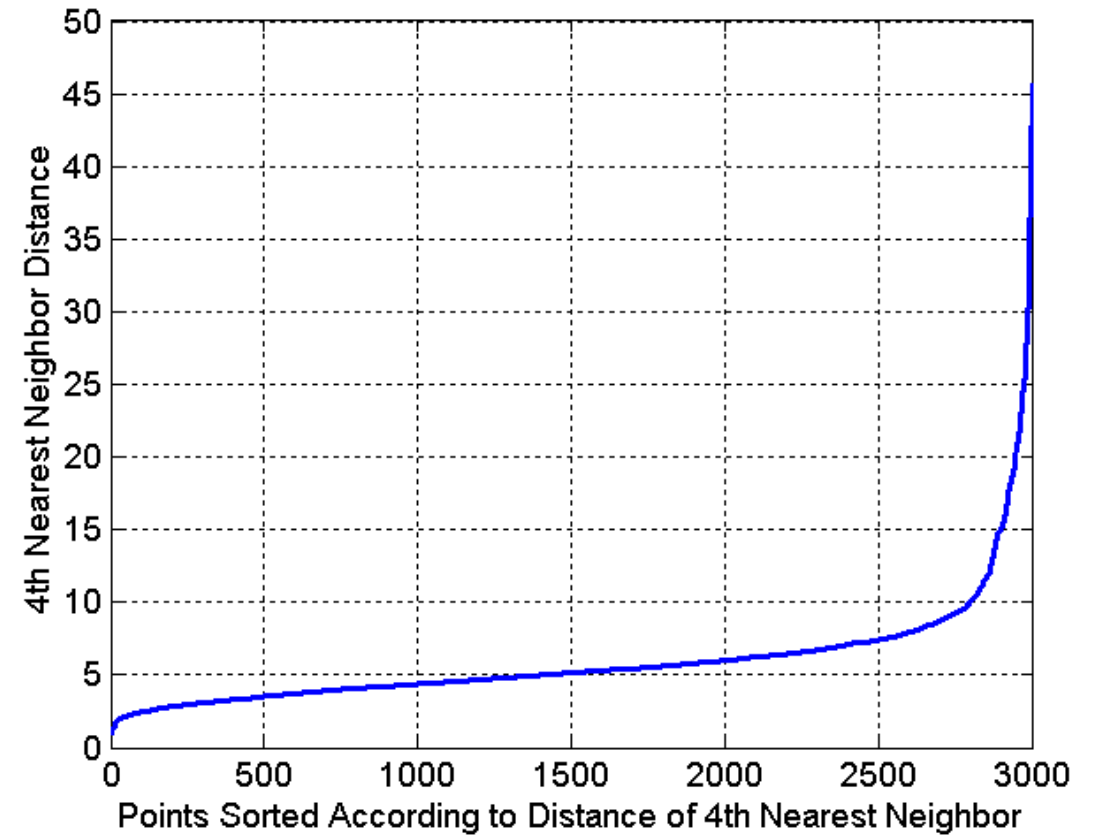
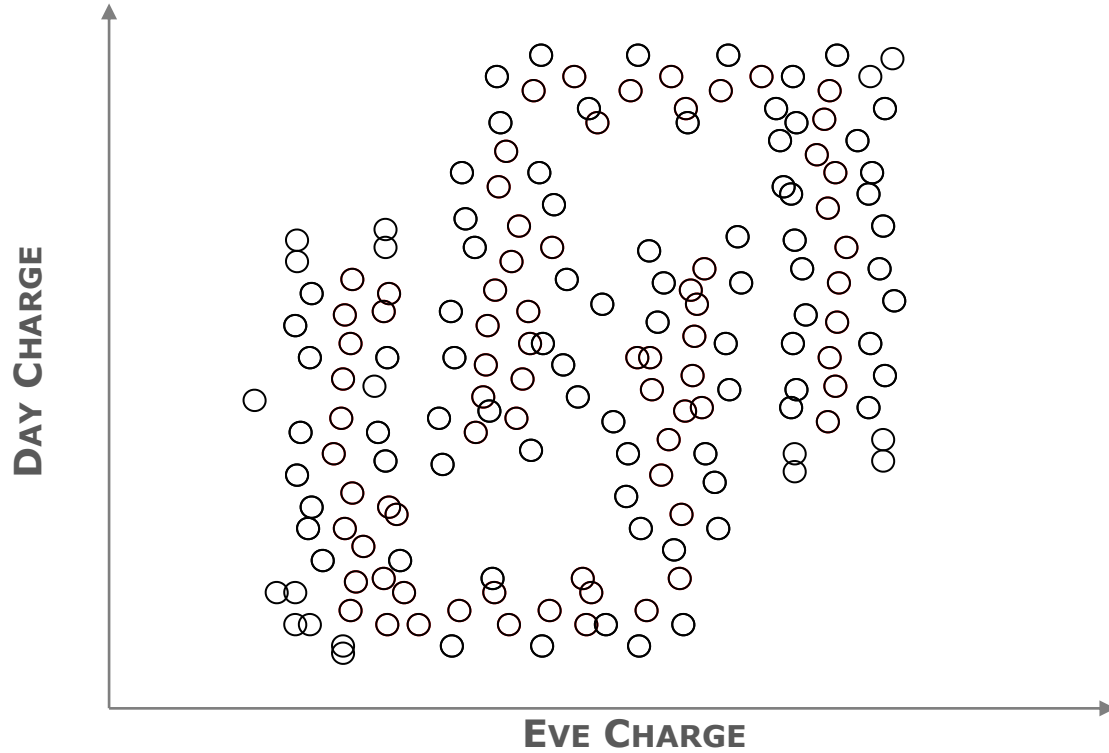


DBSCAN

1. Label all points as core, border, or noise points
2. Eliminate noise points
3. Put an edge between all core points that are within Eps of each other
4. Make each group of connected core points into a separate cluster
5. Assign each border point to one of the clusters of its associated core points



One of the main issues of DBSCAN is the selection of its' parameters, **MINPts** and **Eps**.



We compare DBSCAN to K-Means assuming that there are no ties in distances for either DBSCAN and K-Means, we also assume that DBSCAN always assigns a border point which is associated with more core points to the closest core point.

- ✓ DBSCAN and K-Means assign objects to a single cluster, but K-Means assigns all objects while DBSCAN can discard noise objects
- ✓ DBSCAN can handle clusters of different sizes and shapes and it is not strongly affected by noise or outliers. K-means has difficulties with non-globular clusters and clusters of different sizes. Both algorithms perform poorly when clusters have widely differing densities
- ✓ K-Means can only be used for data that has a well defined centroid, such as mean or median. DBSCAN requires that its definition of density, which is based on the traditional Euclidean notion of density be meaningful for the data
- ✓ K-Means can be applied to sparse, high-dimensional data, such as document data. DBSCAN typically performs poorly for such data because the traditional Euclidean definition of density does not work well for high-dimensional data

- ✓ DBSCAN makes no assumption about the distribution of the data. The basic K-means is equivalent to a statistical clustering approach (Mixture Model) that assumes all clusters come from spherical Gaussian distributions with different means but the same covariance matrix
- ✓ DBSCAN and K-Means both look for clusters using all attributes, that is, they do not look for clusters that may involve only a sub-set of the attributes
- ✓ K-Means can find clusters that are not well separated, even if they overlap, but DBSCAN merges clusters that overlap
- ✓ K-Means has complexity $O(m)$ while DBSCAN is $O(m^2)$ (except for low dimensional Euclidean data.)
- ✓ DBSCAN produces the same set of clusters from one run to another while K-Means, which is typically used with random initialization of centroids, does not
- ✓ DBSCAN automatically determines the number of clusters, for K-Means the number of clusters needs to be specified as a parameter

Many additional Density-based clustering techniques exist that address the issues of efficiency, finding clusters in subspaces, and more accurately modelling density.

They can be grouped in

- ✓ **GRID-BASED CLUSTERING**; they break the data space into grid cells and then form clusters from cells that are sufficiently dense. They can be effective and efficient at least for low-dimensional data (**CLIQUE**)
- ✓ **SUBSPACE CLUSTERING**; they look for clusters (dense regions) in subsets of all dimensions. For a data space with “n” attributes, potentially $2^n - 1$ subspaces need to be searched, and thus an efficient technique is needed to do this
- ✓ **KERNEL DENSITY FUNCTION**; they use kernel density functions to model density as the sum of the influences of individual data objects (**DENCLUE**)